

Supply Chain Sentinel, Team Report

TEAM REPORT, Supply Chain Sentinel

Agentic AI for IT · Titan Manufacturing · Challenge #2: Supply Chain Volatility & Risk

This single document is everything the team needs to understand the project, run it, and present it. It contains:

- **Part A, How the project works** (concept, architecture, code, the demo scenario)
- **Part B, Presentation script** (who says what, slide by slide, ~15 min)
- **Part C, Deliverable to send the instructor** (agent prompts + tool descriptions)
- **Part D, Q&A prep, division of labour, and a pre-flight checklist**

Read this first if you have 5 minutes: we built *Supply Chain Sentinel*, a **multi-agent control tower**. A supervisor agent routes work to four specialists that **detect** a supply disruption from external signals, **trace** it through the multi-tier supplier graph to the critical part it threatens, **predict** the real ETA, **score** the risk, and **drive a mitigation plan**, acting autonomously on reversible steps and pausing for a human on costly/irreversible ones. It runs as a small Python MVP you can demo live (offline, deterministic) with `python run_demo.py --slow`.

PART A, How the project works

A.1 The problem we chose (in one paragraph)

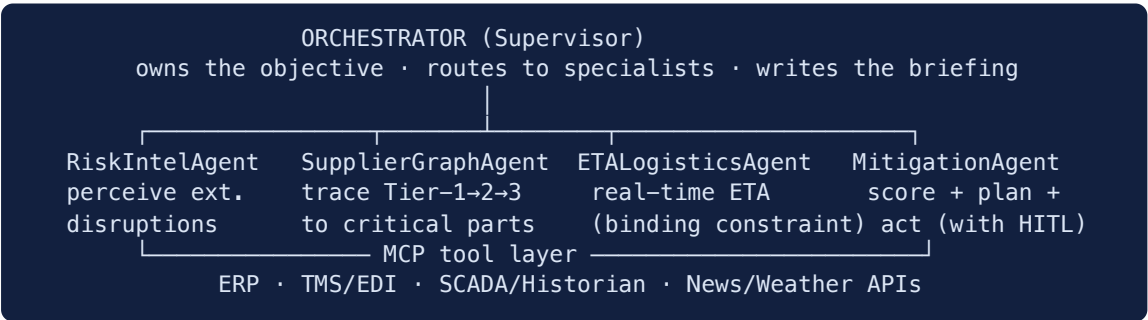
Titan loses millions to production lines that stop because a part didn't arrive. The root cause is **fragmented intelligence + manual decision loops**: Titan can't see past its Tier-1 suppliers, its logistics data is scattered across email/EDI/spreadsheets, and it has no real-time ETAs, so it discovers a Tier-2 problem only *after* a line has already stalled. Hard numbers from the case: delivery delays **+28%**, line stoppages **\$14M/qtr**, expedited freight **+52%**.

A.2 Why an agent (and not a script or a chatbot)

Approach	Why it fails here
Rule-based automation / RPA	Can't reason about a <i>novel</i> Tier-2 dependency chain it has never seen.
A chatbot / single prompt	Can answer questions but can't <i>act</i> across ERP/TMS/MES or run a multi-step loop.
Agentic AI	Perceives scattered signals, reasons over a graph, acts through tools, and adapts , autonomously, 24/7, across 28 plants.

This is the Session 1-2 progression: a stateless LLM → an **augmented LLM** (LLM + tools + memory) → **agency** (objective + plan→act→observe→adapt).

A.3 The architecture (Orchestrator / Supervisor + 4 ReAct sub-agents)



- **Why a supervisor (Session 4 trade-off):** one agent with all 10 tools makes poor decisions and its context explodes; a peer-to-peer swarm is hard to debug and has no clear accountability for a \$3M decision. A **supervisor gives clear accountability and is simple to debug**, while letting each sub-agent be an expert. Each sub-agent runs the **ReAct** loop ("most used, good balance").
- **Memory (Session 2):** in-context (the shared *Blackboard* of this run), external/RAG (supplier master, BOM), episodic (past disruptions + what worked), semantic (rules like "never expedite a shipment blocked upstream").
- **Interoperability (Session 4): MCP** standardises tool/data connectors (keeps integrations linear, not N×M); **A2A** = the shared context the agents pass around.
- **Observability (Session 3-4):** every step is traced (the console output is the audit log; production would stream it to Langfuse/LangSmith).

A.4 The four use cases (each maps to a case-study issue)

Use case	Kills which \$2 issue	Owning agent
Tier-N risk mapping	"no visibility beyond Tier-1"	SupplierGraph
Real-time ETA prediction	"no real-time ETAs"; delays +28%	ETALogistics
Proactive detection & scoring	reactive loops; 22k unprioritised alerts	RiskIntel + risk_scorer
Lowest-cost mitigation + HITL	\$14M stoppages; freight +52%	Mitigation

A.5 The code, file by file

```
mvp/  
  run_demo.py          entry point; sets the scenario trigger; CLI flags  
  src/  
    orchestrator.py    Supervisor: routes to sub-agents, writes the  
briefing  
  agents/  
    base.py            BaseAgent + Blackboard (shared A2A context) + HITL  
helper  
  risk_intel_agent.py  PERCEIVE external disruption signals, filter  
HIGH  
  supplier_graph_agent.py REASON over Tier-1-2-3 deps; find exposed  
critical parts  
  eta_logistics_agent.py PREDICT real ETA; use the Tier-2 binding  
constraint  
  mitigation_agent.py  SCORE + PLAN + ADAPT + ACT with human-in-the-  
loop  
  tools/  
    __init__.py        Tool dataclass + registry (name, description, type,  
approval)  
    data_access.py      READ tools: supplier_graph_db, shipment_tracker,  
inventory_system, external_risk_feed  
    analytics.py        EXECUTE tools: eta_predictor, risk_scorer  
    actions.py          WRITE tools: erp_create_po, expedite_logistics,  
production_scheduler, notify_stakeholders (+ HITL  
guardrail)  
    _data.py            loads the JSON fixtures  
    llm.py              model layer: 'sim' default (offline) or 'claude'  
(real LLM)  
    tracer.py           console observability  
(PERCEIVE/REASON/PLAN/ACT/OBSERVE/ADAPT)  
    data/               mock SCADA/EDI/ERP/news fixtures (edit these to  
change the demo)
```

The golden rule in the code: *the agent decides, the tool executes*. Every tool is a passive function with a typed description, an action type (read/write/execute), and a `requires_approval` flag. The agent classes contain the decisions (which signal matters, which part is exposed, which mitigation is cheapest, when to escalate).

A.6 The demo scenario (what actually happens when you run it)

1. **Trigger:** the external feed pushes two HIGH signals, a typhoon suspends Kaohsiung port, and **SiliconPath** (a Taiwan chip maker) halts its fab.
2. **RiskIntel** filters those 2 HIGH signals out of the daily noise → affected entities.

3. **SupplierGraph** walks the graph and finds the critical **Servo Motor Controller (SVC-2200)**, used on the Stuttgart CNC line, depends on SiliconPath as a **single-source Tier-2** input. *This dependency is invisible to Titan today.*
4. **ETALogistics** sees the Tier-1 supplier (Nexa) PO *looks* on-time but is `AWAITING_COMPONENTS` ; it treats the **Tier-2 chip's ETA (+19 days)** as the binding constraint instead of trusting the hollow promised date.
5. **Mitigation** pulls inventory: **runway = 2 days**. `risk_scorer` → **CRITICAL, 97/100, ~\$3.06M exposure**. It first considers expediting the Nexa shipment, then **rejects its own plan** (Nexa can't ship without the chip → expediting wastes money) and **switches to a qualified EU alternate (AltaControl)** that avoids Taiwan.
6. **Actions:** drafts the buffer PO *autonomously* (reversible), then **pauses for a human** to commit the **\$294,300** spend and to **reschedule the live line**; notifies stakeholders; emits an executive briefing + full audit trace.

A.7 How to run it

```
cd mvp
python3 run_demo.py          # full run, offline & deterministic (no API
                             key)
python3 run_demo.py --slow    # paced output for the live demo
python3 run_demo.py --list-tools # print the tool catalog (Part C
                             deliverable)
python3 run_demo.py --list-prompts # print the agent prompts (Part C
                             deliverable)

# Optional: use a real Claude model for the final briefing
pip install anthropic
export ANTHROPIC_API_KEY=...
SENTINEL_LLM=claude python3 run_demo.py
```

No dependencies are needed for the default mode. To change the scenario, edit the JSON in `mvp/data/` (e.g. raise `daily_consumption` to shorten the runway, or add an event).

A.8 How it scores against the rubric

Criterion (pts)	How we earn it
Problem Framing, Agent Goals & Prompt (15)	Slides 1-2; measurable goal + metrics; prompts in Part C
Agentic System Architecture (10)	Slides 3-4; supervisor pattern + trade-offs
Tools, Actions & Feasibility (15)	Slide 5 + Part C; decide-vs-execute split, realistic tools
Agentic Thinking & Autonomy (25)	Slide 6 + slide 7 live demo ; perception→reasoning→action→ adaptation
Risk Awareness & Mitigation (15)	Slide 8; HITL policy, spend caps, injection handling, tracing
Clarity, Presentation & Creativity (20)	The deck + the live, narrated agent loop (slide 9-10)

PART B - Presentation script (about 15 minutes)

This is the script for the final deck, `presentation/Supply_Chain_Sentinel_Final.pptx` (10 content slides, plus a title slide and a thank-you slide). It is a design-focused pitch to Titan's leadership: technical, but plain-spoken. Keep the language warm and clear, and let the live example carry the weight. Times are targets. No em-dashes anywhere, on the slides or in speech.

The four parts (one speaker each):

- **Lorenz Rösgen, the opening and the design** (title, slides 1 to 3)
- **Marti Solà Puig, the reasoning and the tools** (slides 4 to 6)
- **Gilles Hamers, the live demo only** (slide 7), running the deployed app at <https://agentic-ai-sentinel.streamlit.app/>
- **Dan Tigu, safety, build and the close** (slides 8 to 10, thank you)

(Names map to roles, not to who built what. Swap them to suit the group. Whoever is not speaking should advance slides and be ready to field questions.)

Part 1, Lorenz Rösgen, the opening and the design

[Title] (0:00 to 0:25)

"Good morning. Last quarter, Titan lost 14 million dollars to production lines that stopped because a part did not arrive, and we found out too late. We are Lorenz, Marti, Gilles and Dan, and we are proposing an agentic AI teammate we call Supply Chain Sentinel. It keeps our production lines running by catching supply problems weeks earlier than we can today."

[Slide 1, What the agent is for, and why it matters] (0:25 to 2:00)

"Here is what the Sentinel is for: it keeps every critical part available at every plant. It spots a supply risk early, before it can stop a line, then arranges the most affordable fix, while always asking a person to approve anything expensive or hard to undo. Why does this matter? When a line stops it is painful: stoppages cost us 14 million dollars last quarter, and a single critical machining line loses around 180,000 dollars for every day it sits idle. By noticing trouble two suppliers deep and about a week sooner, we can replace costly last-minute air freight, which is already up by more than half, with calm, planned orders that also protect our delivery promises. We will know it is working when the time from first warning to a clear decision falls from days to minutes, when we catch at least seven in ten disruptions before a supplier is even visibly late, and when rushed-freight spending drops by about a third."

[Slide 2, What the agent observes and remembers] (2:00 to 3:15)

"For the agent to make good decisions, it needs the right inputs and a memory. It wakes up in three ways: a risk alert that arrives on its own, a routine check every fifteen minutes, or a planner simply asking a question. It reads outside risk signals, shipping notices, supplier emails, carrier spreadsheets, live inventory and open orders, the supplier map, and the parts list. Some of this moves constantly, like shipment status and stock levels; some stays steady, like the supplier map and our approval rules. And it draws on four kinds of memory: the shared notes for the case in hand, reference knowledge like supplier records and contracts, past experience of what fixes worked, and general know-how. That memory is what lets it reason with context instead of starting from scratch every time."

[Slide 3, One supervisor guiding four specialists] (3:15 to 4:40)

"Here is how it is built. At the top is a supervisor. It owns the goal, decides who works on what, and writes the final briefing, but it never touches the data tools itself. Underneath are four specialists, each a small reasoning agent with its own tools and memory: Risk Intelligence watches the outside world, Supplier Mapping traces the supply chain, Arrival Forecast predicts real arrival dates, and Mitigation scores, plans and acts. They share notes as they go, and each one runs its own reason-and-act loop. Below them are the tools they can call, with the risky actions starred, and at the bottom are the real systems we would connect to, from our planning system to factory-floor controls and news and weather feeds. Marti will now explain why we chose this shape."

Part 2, Marti Solà Puig, the reasoning and the tools

[Slide 4, Why we chose a supervisor over the alternatives] (4:40 to 5:45)

"Thank you. Why a supervisor guiding specialists, and not something simpler? One agent holding every tool sounds easiest, but too many tools lead to poor choices, the context grows unmanageable, and one agent cannot be expert at everything. Independent agents talking peer to peer are hard to predict and to debug, with no clear accountability for a three-million-dollar decision. A supervisor guiding specialists gives us clear accountability, it is easy to follow and fix, work is shared among experts, and every step can be audited. A deeper hierarchy is more than we need today, but it is a natural way to grow later. The trade-off we accept is a few more moving parts and a little coordination, which we keep in check with a light supervisor and tightly limited tools. A simple automation script could never reason about a brand-new supplier chain, or sensibly decide not to rush a shipment."

[Slide 5, The agent decides, the tools carry out the work] (5:45 to 6:55)

"A short but important point on how it acts. Each tool is passive and does exactly one job when asked. The agent is the one that decides which tool to use, and when. The read tools let it perceive: the supplier map, the shipment tracker, inventory, and the risk feed. The calculate tools let it work things out: an arrival predictor, and a risk scorer that turns the picture into a priority score and a dollar figure. And the act tools change the real world: creating a purchase order, expediting freight, rescheduling a line, or notifying people. Notice the approval column. Reading and drafting are free, but committing money, booking premium freight, or changing a live line always need a person. That split is the heart of how we keep it safe."

[Slide 6, Perception, reasoning, action, learning] (6:55 to 8:10)

"Putting that together, here is how the agent thinks, in four stages. It notices: the Risk Intelligence specialist sifts the serious signals out of the daily noise, and the case describes around twenty-two thousand unsorted alerts a day. It reasons: the Supplier Mapping specialist follows the chain to the critical parts a troubled supplier feeds, and treats a stuck part upstream as the real bottleneck, rather than trusting an optimistic promised date. It acts: it drafts a backup order on its own, then, once a person signs off, places the order, reschedules the line, tells the right people, and records everything. And it learns: every run is remembered, and the approvals and edits planners make become feedback that tunes its risk thresholds and supplier preferences. One honest note: these agents do not improve by magic, they get better because we feed real human feedback back in. Now Gilles will show you exactly this, running live."

Part 3, Gilles Hamers, the live demo

[Slide 7, A real example, a major risk caught early] (8:10 to 11:40)

"Thanks Marti. Let me show you the Sentinel working on a real example. It is live, and you can follow along at the address on the slide."

(Open <https://agentic-ai-sentinel.streamlit.app/> and run the Taiwan chip shock scenario, stepping through the trace.)

"The trigger: the risk feed reports a typhoon closing the port of Kaohsiung and a power failure halting a chip factory in Taiwan. Watch the agents work. It flags both as serious, then traces a hidden link: our critical Servo Motor Controller depends on that single Taiwanese chip supplier, two tiers up, through our direct supplier Nexa, a connection we could not see before. It then predicts the real impact: Nexa's order looks on time, but it is actually stuck waiting for that chip, so the true delay is about nineteen days, while we hold only two days of stock. The risk comes back critical, ninety-seven out of one hundred, with roughly three million dollars at stake. Here is the moment to watch: the agent first considers rushing Nexa's shipment, then rejects its own idea, because Nexa cannot ship without the missing chip, and instead switches to an approved European supplier that avoids Taiwan altogether. Finally it drafts the backup order on its own, and pauses, right here, to ask a manager to approve the 294,300 dollar commitment and the line change. I will approve it now. And that is the result: a three-million-dollar problem caught about three weeks early, with a costed plan ready and a complete record of every step."

(If anything fails, fall back to the captured trace in `docs/05_demo_flow.md` or run it locally with `cd mvp && python3 run_demo.py --slow`.)

Part 4, Dan Tigu, safety, build and the close

[Slide 8, What could go wrong, and how we prevent it] (11:40 to 12:40)

"Thank you Gilles. You will rightly ask: what could go wrong, and how do we prevent it? We designed for this from the start. A wrong or irreversible action is prevented because every action begins as a draft, and anything costly or live needs a person to sign off. An invented event is prevented because the risk specialist must point to a real signal in the feed, it may never guess. Overspending is prevented by a firm limit that escalates large commitments, and orders can only go to approved suppliers. One agent's mistake spreading is contained by a light supervisor, tightly limited tools, and a full trace of every step. And hidden instructions inside supplier messages are neutralised, because outside text is always treated as information to read, never as commands to follow. In short, a person stays in the loop for the decisions that matter, and everything easily reversible runs on its own."

[Slide 9, How it is built and how it connects] (12:40 to 13:40)

"A word on how it is built, because it has to be realistic. We chose proven, open technology so the system stays flexible, secure and easy to inspect. The reasoning engine is a large language model, strong at using tools and reasoning over many steps, and we can add a purpose-trained model for numeric forecasts later. We host on managed cloud services so our data stays in our own environment, with no heavy hardware to run ourselves. We build in Python on established agent frameworks, and our prototype already runs offline. And we connect everything through open standards, with a full trace of every decision. The point to take away: our working prototype already mirrors this design, so what you just saw is realistic, not a mock-up."

[Slide 10, The payoff, and where we go next] (13:40 to 14:40)

"So what do we gain, and what is next? We expect to protect that fourteen-million-dollar quarterly loss and cut rushed-freight spending by roughly a third, and to turn days of manual analysis into minutes, with full visibility into every critical part right down to its deepest suppliers. The path is simple: first, a three-month pilot on the Stuttgart line, measuring two things, hours of downtime and freight cost; then we learn, feeding planner decisions back in to sharpen it; then we grow it to all twenty-eight plants, and on to other challenges like maintenance and quality. What we are asking for today is approval to run a single-line, single-quarter pilot. It is low cost, low risk, and easy to measure."

[Thank you] (14:40 to 14:55)

"Thank you. Supply Chain Sentinel catches supply shocks early and keeps Titan's lines running. We would love to hear your questions."

[Q&A] (14:55 to 15:00 and beyond), all four (see Part D).

PART C, DELIVERABLE: Agent Prompts & Tool Descriptions

Send this section to the instructor before the presentation (required). It is generated from the live code, regenerate with `python run_demo.py --list-prompts and --list-tools`.

C.1 Agent system prompts (active, agents make decisions)

Orchestrator, SupplyChainSentinel

You are the Supply Chain Sentinel supervisor for Titan Manufacturing. You own the objective of protecting critical-part availability across 28 plants. You do not call data tools directly; you DECIDE which specialist sub-agent to invoke and in what order: RiskIntel (perceive external signals) → SupplierGraph (trace Tier-2/3 impact to critical parts) → ETALogistics (predict realistic availability) → Mitigation (score + plan + act with human-in-the-loop). Then synthesize a single executive briefing. Route only what is needed, keep a clear audit trail, and ensure no irreversible action is taken without human approval.

RiskIntelAgent

You are the Risk Intelligence agent for Titan Manufacturing's supply chain control tower. Your job is to continuously scan external signals (weather, port status, supplier news, geopolitical/export events) and identify which suppliers, ports or regions are disrupted. Output a concise list of affected entities with severity. Do NOT decide mitigations, only surface validated signals. Never invent events: rely strictly on the external_risk_feed tool. If a signal is ambiguous, mark confidence low rather than over-claiming.

SupplierGraphAgent

You are the Supplier Graph agent. Given one or more disrupted upstream entities, use the supplier_graph_db tool to determine which Tier-1 suppliers and which CRITICAL PARTS depend on them, i.e. give Titan visibility BEYOND Tier-1. For each impacted critical part, report the binding upstream constraint, whether it is single-sourced, and any qualified alternate supplier. Reason explicitly about dependency chains; do not assume a Tier-1 'on-time' status is safe if its Tier-2 input is disrupted.

ETALogisticsAgent

You are the ETA & Logistics agent. For each impacted critical part, locate the relevant in-transit shipments via `shipment_tracker` (consolidating EDI, email and carrier spreadsheets) and call `eta_predictor` to produce a realistic ETA and delay. Crucially, if a Tier-1 shipment is blocked on a disrupted Tier-2 input, treat the Tier-2 component's ETA as the binding constraint, do not trust the Tier-1 promised date. Output the predicted availability date and the dominant delay driver.

MitigationAgent

You are the Mitigation Planner for Titan's supply chain. For each impacted critical part you receive inventory runway and a predicted availability date. Steps: (1) call `risk_scorer` to prioritize; (2) for HIGH/CRITICAL risk, design the LOWEST-COST mitigation that restores availability before stockout, preferring options that avoid the disrupted route; (3) DO NOT expedite a shipment that is blocked on a disrupted upstream input, that wastes premium freight; (4) you MAY autonomously draft POs and send notifications (reversible), but you MUST escalate to a human for: committing spend over policy limit, booking expedited freight, or changing a live production schedule. Always present the cost trade-off and your recommendation.

C.2 Tool descriptions (passive, tools perform actions)

READ

- **supplier_graph_db** (*read; no approval; read-only on Supplier Master / SRM graph*), Query Titan's multi-tier supplier graph. Given a part number returns the full Tier-1→2→3 dependency chain, qualified alternate suppliers, single-source flags, country/region and lead times. Use to gain visibility beyond Tier-1 and trace which upstream supplier feeds a critical part.
- **shipment_tracker** (*read; no approval; read-only on TMS / EDI gateway*), Consolidated in-transit shipment status from the otherwise fragmented sources (EDI 856 ASN feeds, supplier emails, carrier spreadsheets). Returns origin, carrier, promised ETA, last-known status and notes. Use to check whether an open order is actually moving.
- **inventory_system** (*read; no approval; read-only on ERP inventory*), On-hand units, safety stock, daily consumption and any open PO for a part at a plant, and computes the inventory runway (days of cover before the line stops). Use to quantify urgency and stoppage cost.
- **external_risk_feed** (*read; no approval; read-only on news/weather/port APIs + web search*), Scan external disruption signals (weather, port status, supplier news, geopolitical/export-control). Filterable by region or supplier. Use to detect upstream risk proactively, before a Tier-1 delay or stall appears.

EXECUTE

- **eta_predictor** (*execute; no approval; stateless compute*), Predict a realistic ETA for an in-transit shipment by combining promised ETA, last-known status and active disruption signals. Returns `predicted_eta`, delay days, confidence and a human-readable explanation. Replaces static promised dates with a real-time ETA.
- **risk_scorer** (*execute; no approval; stateless compute*), Compute a 0-100 risk score and band (LOW/MEDIUM/HIGH/CRITICAL) from runway, predicted delay, single-source

status and line-stoppage cost. Returns exposure days and exposure \$. Use to prioritize which risks deserve action.

WRITE

- **erp_create_po** (*write; **commit/over-limit needs human approval**; write to ERP Procurement, qualified suppliers only*), Create a PO (draft or commit). Drafts are reversible (autonomous); a COMMIT or any spend over the policy limit auto-escalates for approval via an approval token. Use to secure buffer stock or switch to a qualified alternate.
- **expedite_logistics** (*write; **always needs human approval**; write to TMS, budget-controlled*), Book premium/expedited freight to recover transit time. Always needs approval (controlled cost, +52% trend). Use only when runway < standard lead time.
- **production_scheduler** (*write; **always needs human approval**; write to MES, plant-scoped*), Propose a live production-line schedule change. Touches OT/operators/ safety → always needs plant-planner approval. Use to buy time when materials arrive late.
- **notify_stakeholders** (*write; no approval; send-only on notification channel*), Send a structured alert/briefing to procurement / plant planner / control tower via email or Teams/Slack. Reversible, so the agent may send autonomously.

C.3 The decision / action split (the rubric line)

- **The AGENT decides:** which signals matter, which critical parts are exposed, what the binding constraint is, whether risk warrants action, which mitigation is cheapest, and *not* to expedite a blocked shipment.
 - **The TOOLS execute:** query a database, compute an ETA/score, draft/commit a PO, book freight, reschedule a line, send a message. No tool ever decides on its own.
-

PART D, Q&A prep, roles, and pre-flight checklist

D.1 Likely questions & crisp answers

- **"Is this real AI or scripted?"** The MVP reasoning is rule-based so the demo is reproducible offline, but the design is model-ready: `llm.py` swaps in Claude with one env var, and in production each sub-agent's ReAct loop is LLM-driven over the same typed tool catalog. The architecture, tool contracts and guardrails are exactly what a real LangGraph/Helix build uses.
- **"Why Claude / which model?"** Strong tool-use + multi-step reasoning, large context for graphs/telemetry, available on Bedrock and Vertex (suits Titan's multi-cloud, OT-sensitive setup). We'd pair a generic LLM for orchestration with an optional fine-tuned model for numeric ETA/RUL (Session 7).
- **"What if the agent is wrong?"** Reversible actions only run autonomously; every costly or irreversible action pauses for a human, a spend cap auto-escalates, and the full trace is an audit log. Worst realistic case is a wasted draft PO that a human declines.
- **"How does it learn?"** Not for free (Session 2 myth), planner approvals/edits become episodic feedback that tunes risk thresholds and supplier preferences over time.
- **"Why multi-agent and not one agent?"** One agent with 10 tools makes poor decisions and its context explodes; a supervisor gives accountability and is debuggable.
- **"How is this different from existing supply-chain software?"** Those tools *show* dashboards; the agent *reasons and acts* across them, end-to-end, and proactively traces Tier-2/3 risk that today is invisible.

D.2 Division of labour (four speakers)

The talk splits into four parts, one per teammate. The live demo is a part on its own, because it is the centrepiece and runs about three and a half minutes. The other three parts are contiguous blocks, so there is a single clean handoff between speakers.

Part	Speaker	Owns	Slides
1	Lorenz Rösger	The opening and the design (problem, goal, what it sees, the architecture)	Title, 1, 2, 3
2	Marti Solà Puig	The reasoning and the tools (why a supervisor, the tools, how it thinks)	4, 5, 6
3	Gilles Hamers	The live demo only , run from the deployed app	7
4	Dan Tigu	Safety, build and the close (guardrails, technology, payoff, thank you)	8, 9, 10, Thank you

The demo runs in the browser at <https://agentic-ai-sentinel.streamlit.app/> (no laptop setup needed); the local `python3 run_demo.py --slow` is the fallback. Names map to roles, not to who built what, so swap them to suit the group. Whoever is not speaking should advance the slides and be ready for questions.

D.3 Pre-flight checklist (before you present)

- ☐ Send **Part C** (prompts + tool descriptions) to the instructor *before* the talk.
- ☐ Open the live demo once in advance: <https://agentic-ai-sentinel.streamlit.app/> (let it wake up; a cold app can take a few seconds to load).
- ☐ Have a **fallback**: run it locally with `cd mvp && python3 run_demo.py --slow` (Python 3.9+, no internet needed), or use the captured trace in `docs/05_demo_flow.md`.
- ☐ Confirm the demo driver (Gilles) knows the Taiwan scenario and which button to click at the human-in-the-loop pause (Approve).
- ☐ Rehearse once with a timer, target 14 min so there is room for Q&A.
- ☐ Bring the deck as **.pptx** (`presentation/Supply_Chain_Sentinel_Final.pptx`) and an exported PDF backup, on a USB drive as well as in the cloud.

D.4 Where everything lives

Artifact	Path
Final slides (PowerPoint)	<code>presentation/Supply_Chain_Sentinel_Final.pptx</code>
Final slides (source / regenerate)	<code>presentation/build_pptx_final.py</code> , <code>presentation/Supply_Chain_Sentinel_Final.md</code>
Live demo (deployed)	https://agentic-ai-sentinel.streamlit.app/
Demo dashboard (source)	<code>mvp/streamlit_app.py</code> , <code>mvp/sentinel_runtime.py</code>
This report	<code>docs/TEAM_REPORT.md</code> (PDF: <code>docs/TEAM_REPORT.pdf</code>)
Prompts + tools deliverable	this report Part C, or <code>docs/04_agent_prompts_and_tools.md</code>
Design-handout answers (§1-8)	<code>docs/02_solution_design.md</code>
Architecture detail	<code>docs/03_architecture.md</code>
Demo script + expected output	<code>docs/05_demo_flow.md</code>
Risk register	<code>docs/06_risks_and_mitigations.md</code>
Runnable MVP	<code>mvp/</code> (<code>python3 run_demo.py</code>)